Implementing IPv6 Segment Routing

David Lebrun
<david.lebrun@uclouvain.be>

UCLouvain

Netdev 1.2, Tokyo, October 2016

Table of Contents

Segment Routing

Implementation

Network Function Virtualization

Conclusion

Table of Contents

Segment Routing

Implementation

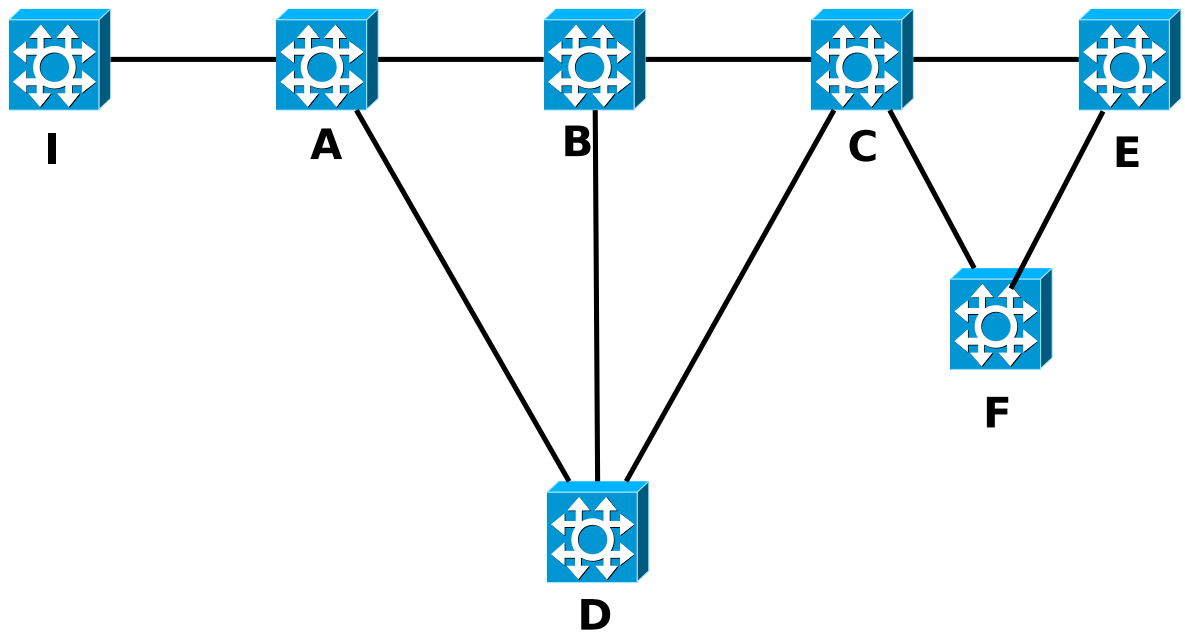
Network Function Virtualization

Conclusion

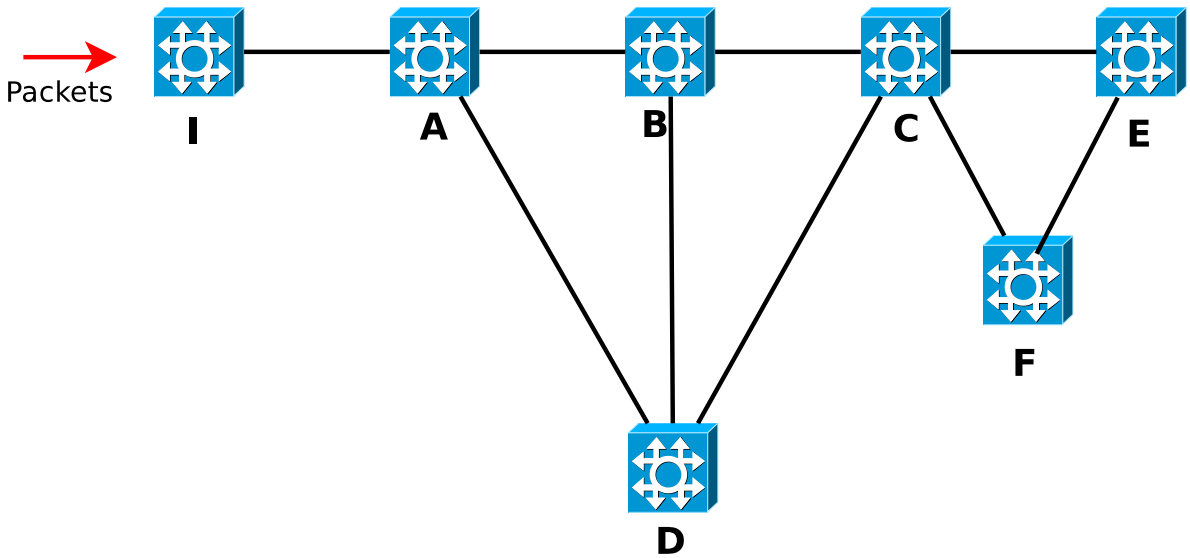
Segment Routing

- Source routing paradigm
- State embedded in packet (*segments*)
- Segments: node, adjacency, service
- IETF draft-spring-segment-routing-09

Segment Routing: illustration (1)

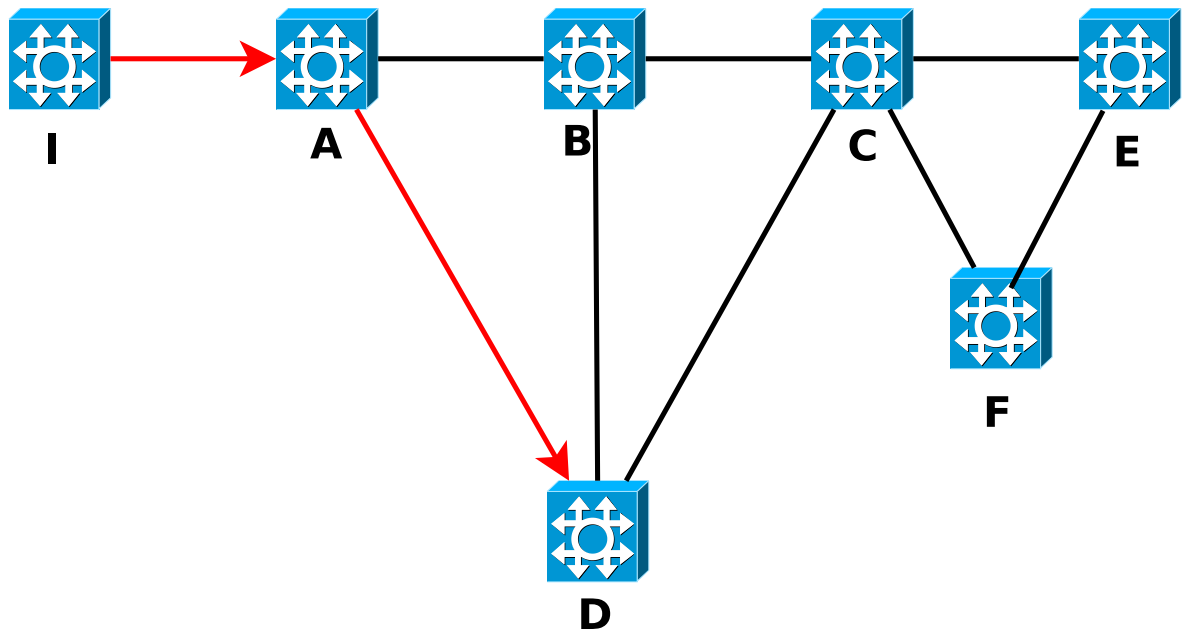


Segment Routing: illustration (2)



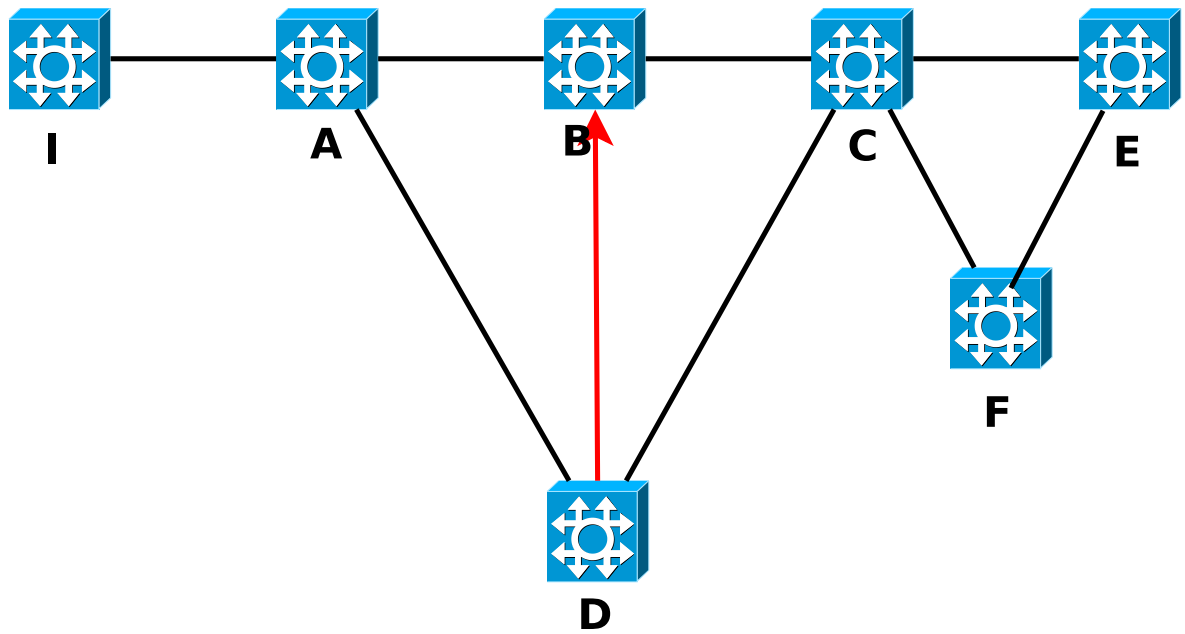
- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[0] (SD)`

Segment Routing: illustration (3)



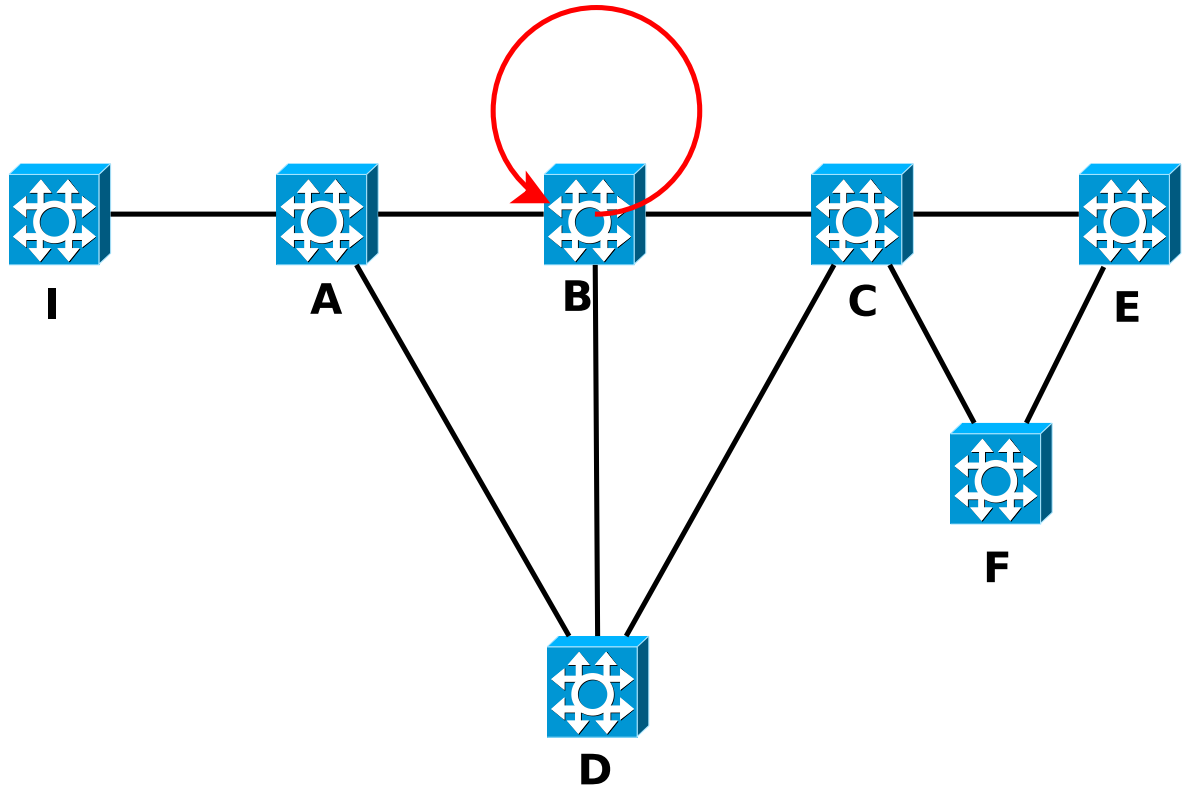
- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[0] (SD)`

Segment Routing: illustration (4)



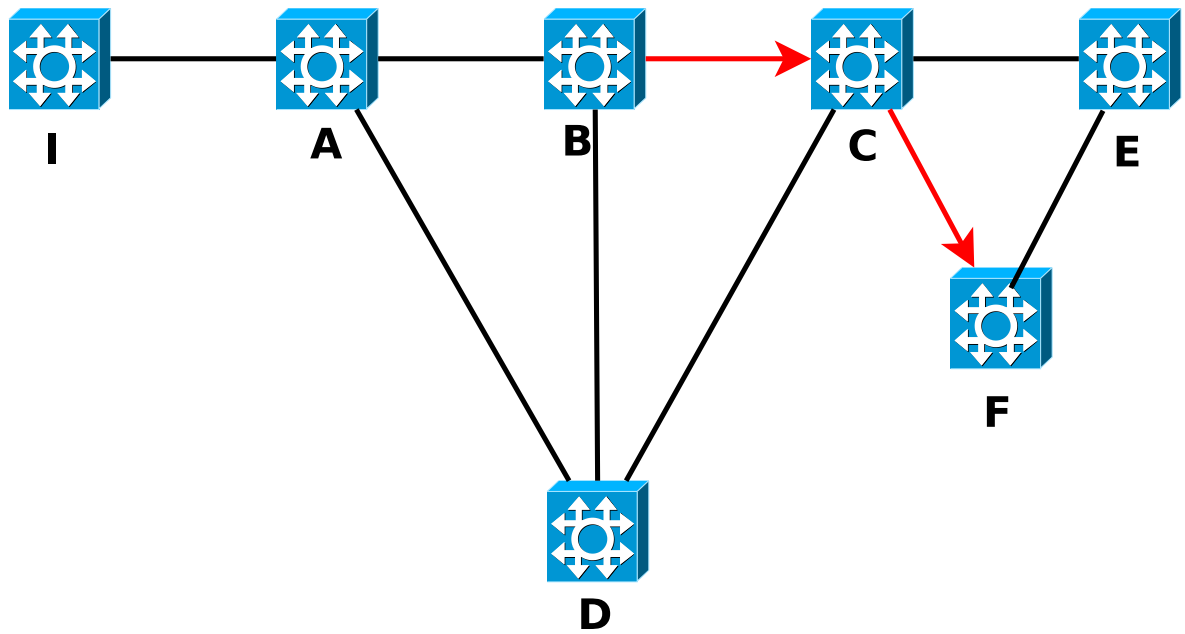
- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[1] (SB)`

Segment Routing: illustration (5)



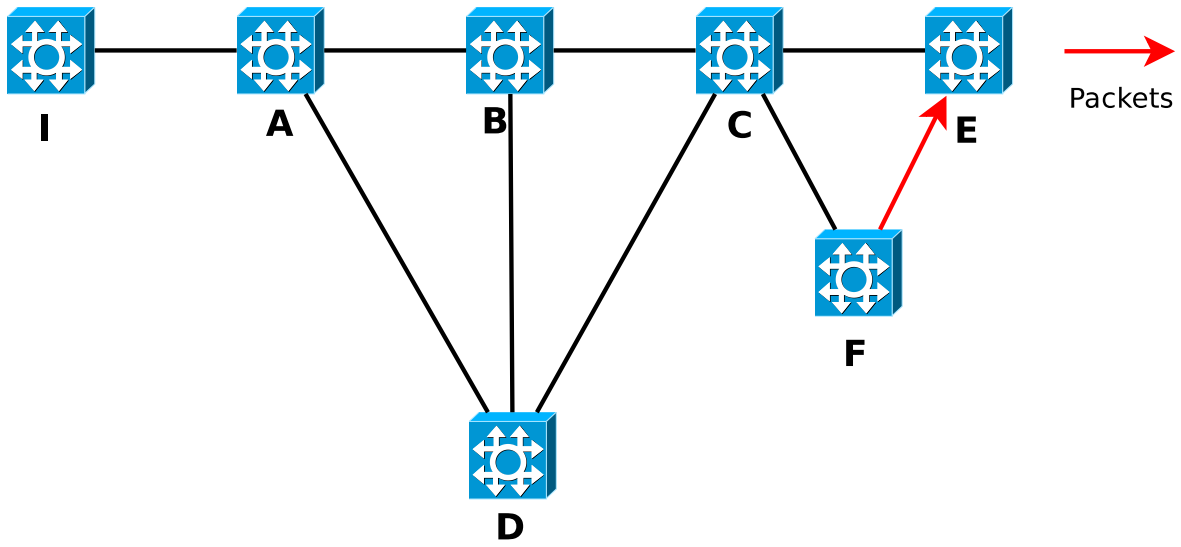
- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[2] (SS)`

Segment Routing: illustration (6)



- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[3] (SF)`

Segment Routing: illustration (7)

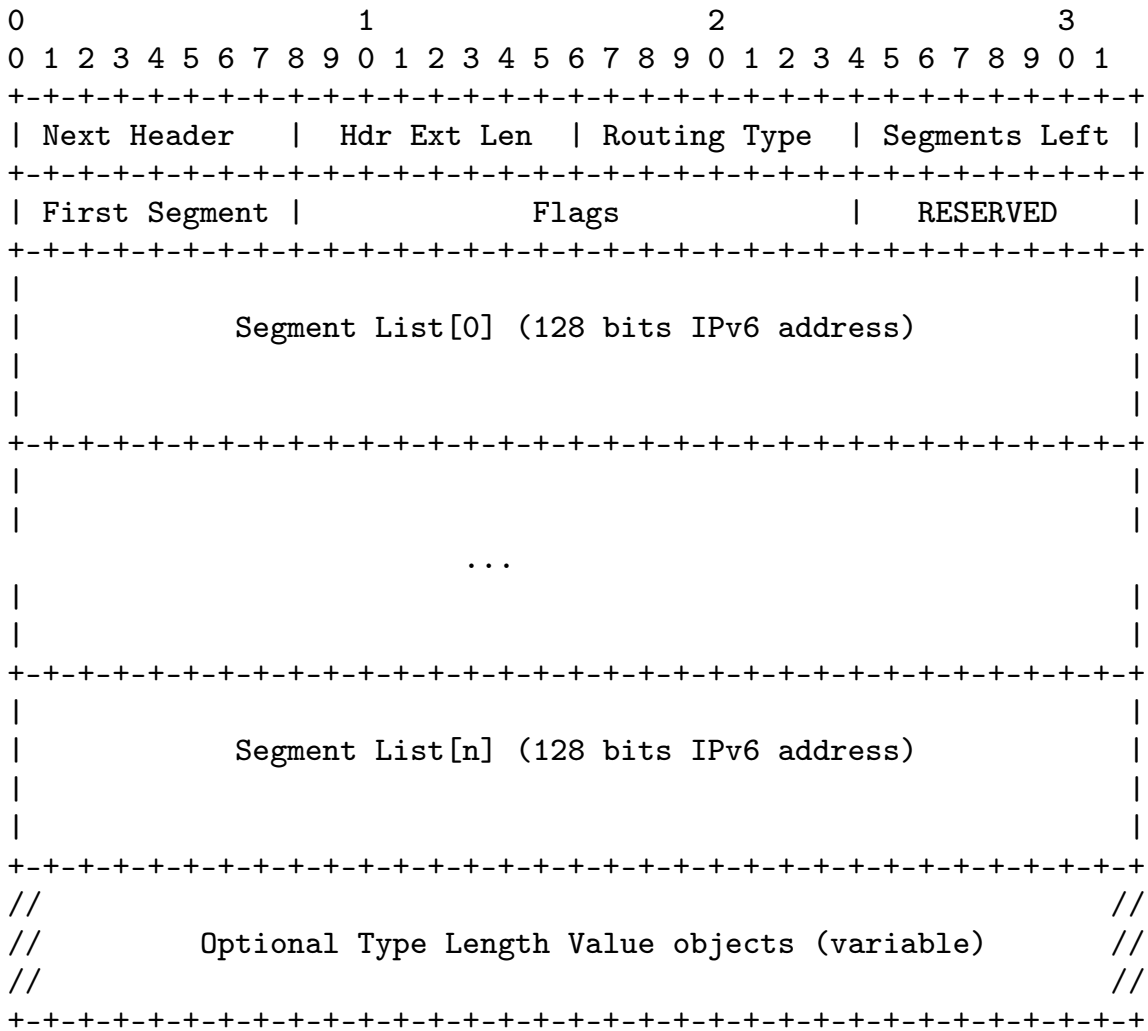


- Abstract SR Header
  - `Segments = SD, SB, SS, SF, SE`
  - `Ptr = Segments[4] (SE)`

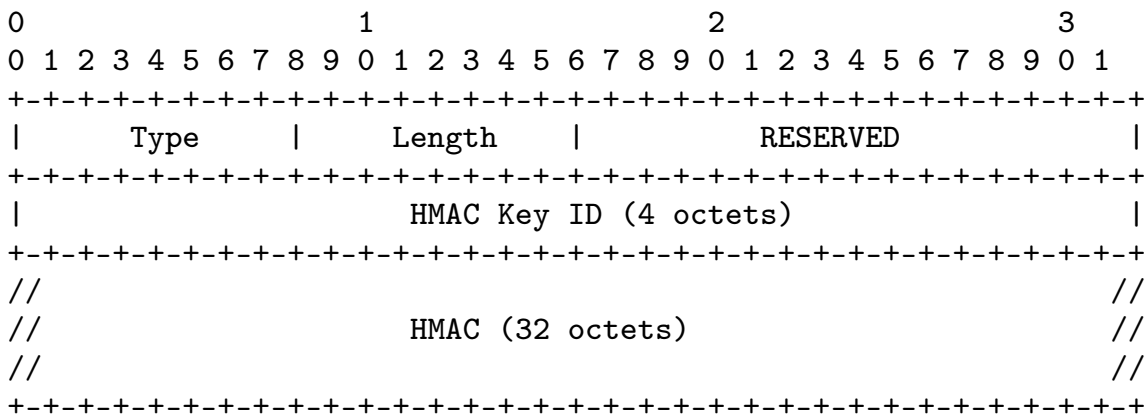
SR flavors

- SR-MPLS (*segment*: 20-bit label)
- SR-IPv6 (*segment*: 128-bit IPv6 addr)
  - IPv6 Routing Header extension
  - HMAC TLV for authenticity & integrity

SR-IPv6

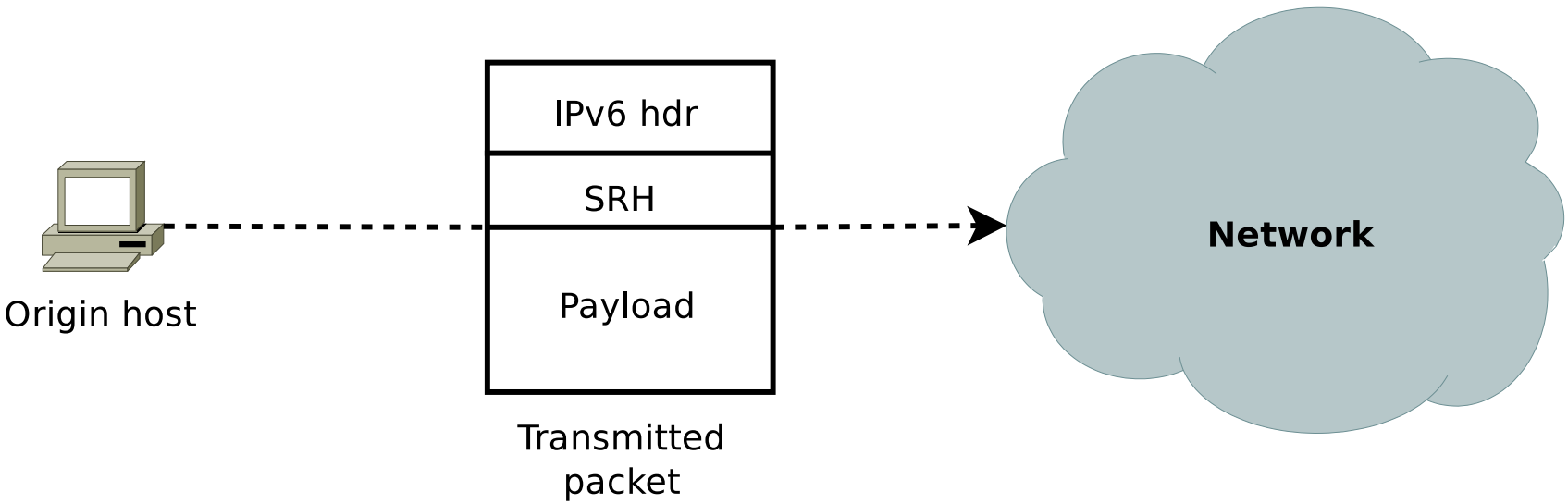

0 1 2 3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
Next Header
Hdr Ext Len
Routing Type
Segments Left
First Segment
Flags
RESERVED
Segment List[0] (128 bits IPv6 address)
...
Segment List[n] (128 bits IPv6 address)
Optional Type Length Value objects (variable)

HMAC TLV


0 1 2 3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
Type
Length
RESERVED
HMAC Key ID (4 octets)
HMAC (32 octets)

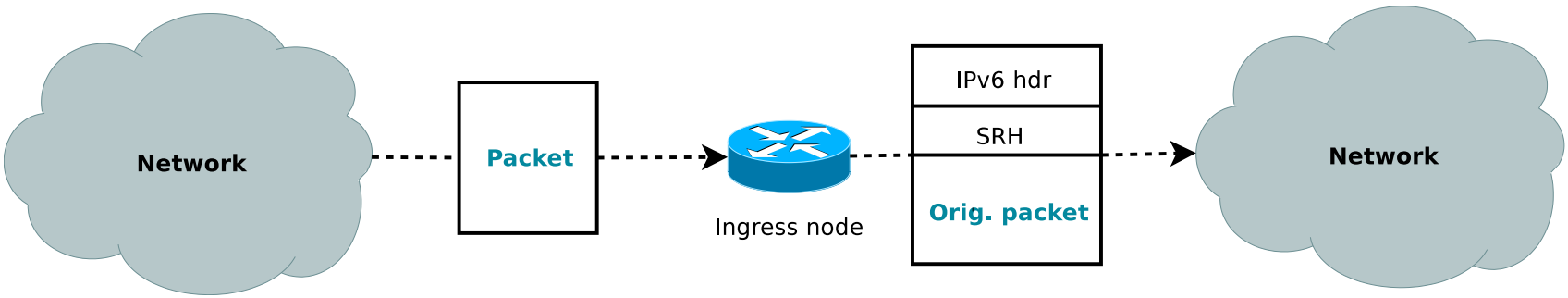
SR-IPv6 operations (1)

- Origin SR host: push SRH in packet build



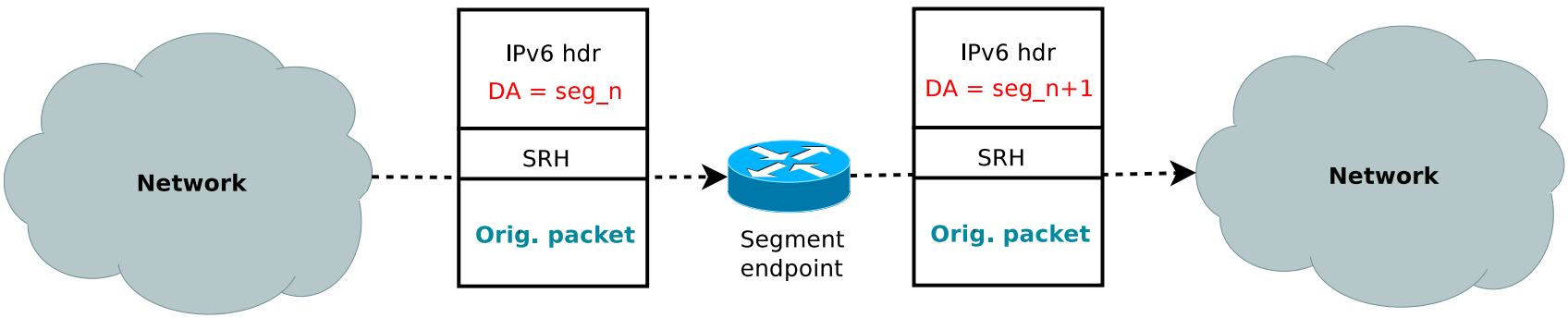
SR-IPv6 operations (2)

- SR ingress: encapsulate with outer IPv6 header + SRH



SR-IPv6 operations (3)

- Segment endpoint: route packet to next segment



SR-IPv6 operations (4)

- SR egress: decapsulate packet and route inner packet to DA

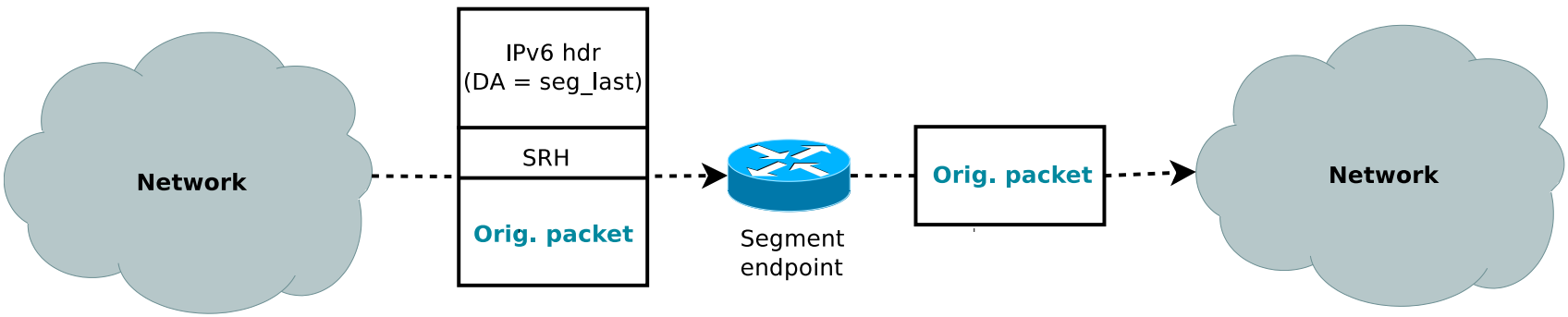


Table of Contents

Segment Routing

Implementation

Network Function Virtualization

Conclusion

Features

- SR-enabled packet processing (data plane)
- SRH insertion/encapsulation (control plane)
- HMAC support (control + data plane)

Data plane part (1)

- `net/ipv6/exthdrs.c`
- Processing of Routing Header type 4
- Decrement `segments_left` index
- Update `ipv6_hdr(skb)->daddr` with next segment
- Call `ip6_route_input(skb)`

Data plane part (2)

- SRH decapsulation (egress)
- pull, reset headers, `netif_rx`
- `tcpdump` shows the packet twice

Control plane part (1)

- `net/ipv6/seg6.c net/ipv6/seg6_iptunnel.c`
- **SRH encapsulation (ingress)**
- Lightweight tunnels support
- `ip -6 route add` *`prefix`* `via` *`gw`* `encap seg6 mode encap segs` *`seg1,seg2,seg3`*

Control plane part (2)

- `net/ipv6/ipv6_sockglue.c net/ipv6/exthdrs.c`
- **Per-socket SRH insertion (endhost)**
- Pushed by `ipv6_push_nfrag_opts`
- Set with `setsockopt(fd, IPPROTO_IPV6, IPV6_RTHDR, ...);`

Insertion vs encapsulation

- Missing support for IPv6 extension headers in drivers
- `ixgbe 0000:0b:00.1: partial checksum but l4 proto=2b!`
- IPv6 encapsulation works well incl. w/ GRO

HMAC part (1)

- `net/ipv6/seg6_hmac.c`
- Calls in `net/ipv6/exthdrs.c` and `net/ipv6/seg6_iptunnel.c`
- Validate SR-enabled packets with HMAC signature
- Generate valid signature for locally imposed SRHs

HMAC part (2)

- $HMACKeyID \Rightarrow < algo, secret >$
- Controlled through `genetlink` interface
- Interface sysctl `seg6_require_hmac`

Table of Contents

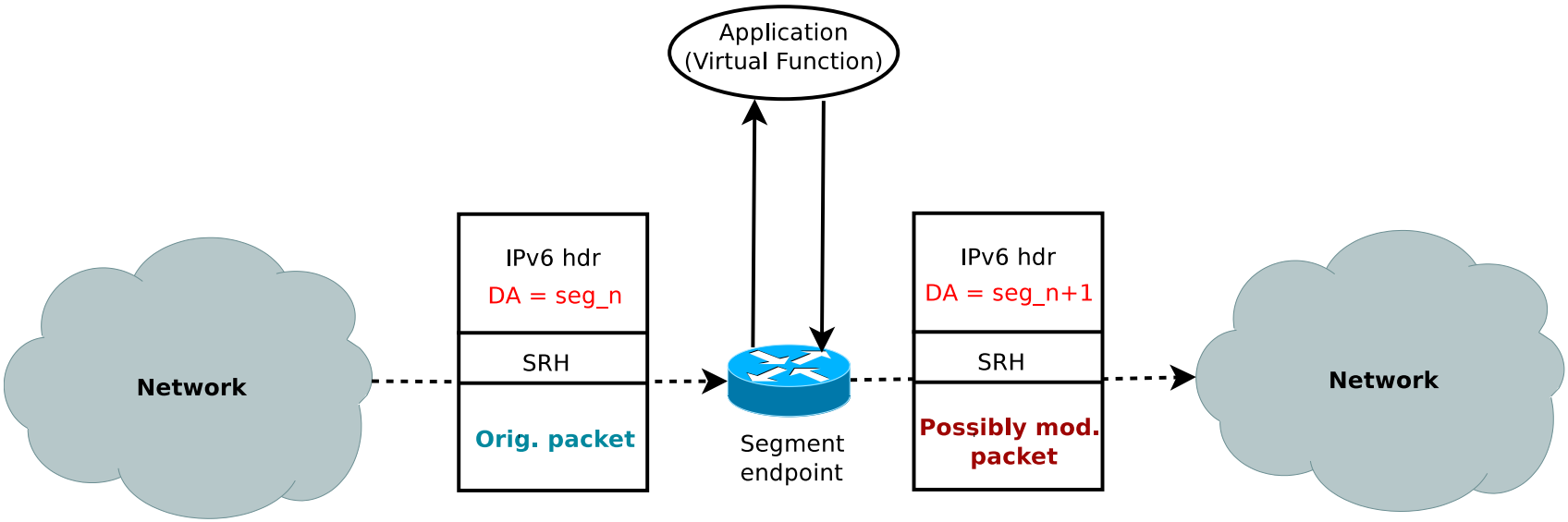
Segment Routing

Implementation

Network Function Virtualization

Conclusion

NFV with SR


Application
(Virtual Function)
Network
IPv6 hdr
DA = seg_n
SRH
Orig. packet
Segment
endpoint
IPv6 hdr
DA = seg_n+1
SRH
Possibly mod.
packet
Network

NFV-SR requirements

- Fast, generic kernel-to-app packet transfer
- Packet modifiable by app
- App-to-kernel packet transfer

NFV-SR attempted solution

- mmap-ed netlink tx/rx rings
- Buffering, reordering issues
- Not GRO-friendly
- Very specific
- Removed from mainline

NFV-SR possible solutions

- `nfqueue` w/ `NF_REPEAT` ?
- GRO-friendliness issue
- Interface-based solution ?
- Suggestions welcome

Table of Contents

Segment Routing

Implementation

Network Function Virtualization

Conclusion

Conclusion

- Draft fully implemented
- Need driver support for IPv6 extension headers
- Need packet transfer mechanism for NFV support

Conclusion

- Thanks to Cisco and ARC-SDN grant from Fédération Wallonie-Bruxelles for funding

- Patch submission by end of October

- Contact: david.lebrun@uclouvain.be